



UNIVERSIDADE
FEDERAL DO CARIRI

Superfícies visíveis

Professora Dra. Luana Batista da Cruz
luana.batista@ufca.edu.br

Roteiro

01 Introdução

02 Back face culling

03 Algoritmo Z-Buffer

01

Introdução

Classificação dos algoritmos

Introdução

- Rendering de polígonos
 - Por eficiência, só queremos renderizar as faces poligonais que são visíveis para a câmera
 - Existem diversos algoritmos para **detecção de superfícies visíveis** (ou eliminação de superfícies ocultas) que variam conforme
 - Complexidade da cena
 - Tipo de objeto desenhado (convexo, côncavo...)
 - Custo computacional, etc

Classificação dos algoritmos

- Os algoritmos existentes podem ser classificados em dois grandes grupos
 - **Métodos de espaço de objeto:** compara objetos entre si ou partes de objetos (lados), para determinar a visibilidade. É realizado antes da projeção
 - **Métodos de espaço de imagem:** compara pixel por pixel no plano de projeção para determinar a visibilidade

Classificação dos algoritmos

- Discutiremos dois algoritmos para visibilidade
 - **Back-face culling:** um método de espaço de objeto simples e rápido para descartar as faces traseiras de um poliedro
 - **Z-buffer:** um método de espaço de imagem comumente usado para detectar superfícies visíveis, que compara profundidades das superfícies para cada pixel projetado no plano de projeção

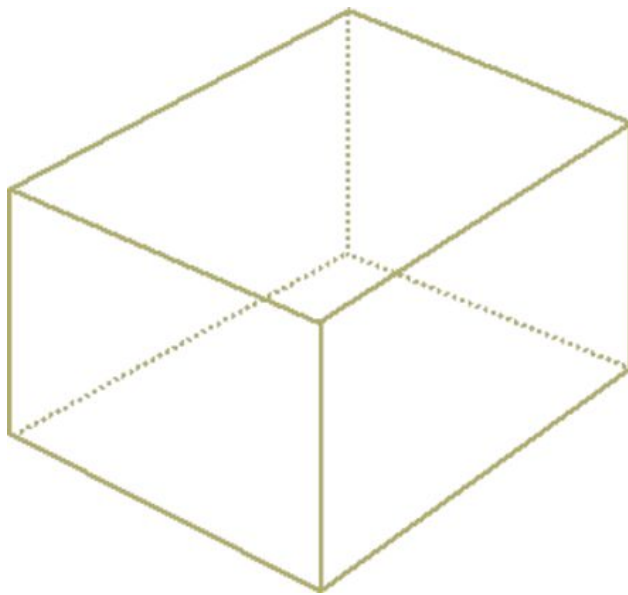
02

Back face culling

Exemplos

Back face culling

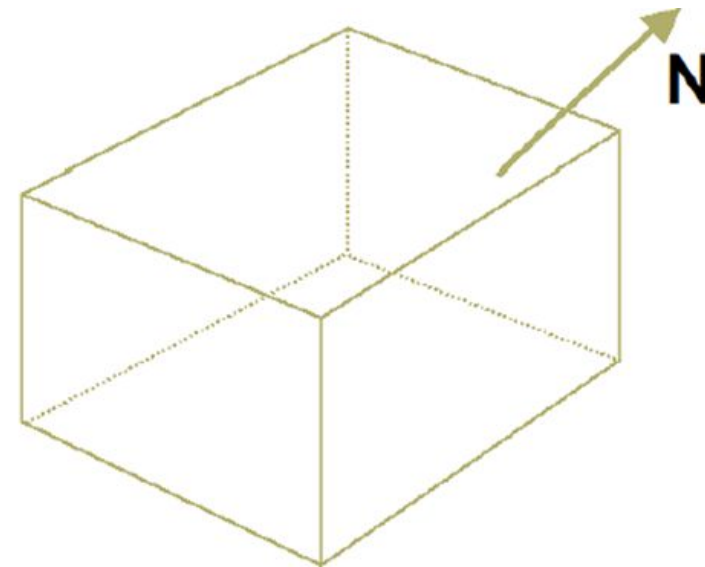
- Se faces pertencem a um objeto sólido (cubo, por exemplo), não é necessário renderizar as faces de trás (não visíveis)



- Apenas 3 faces precisam ser traçadas
- Faces de trás podem ser removidas do pipeline

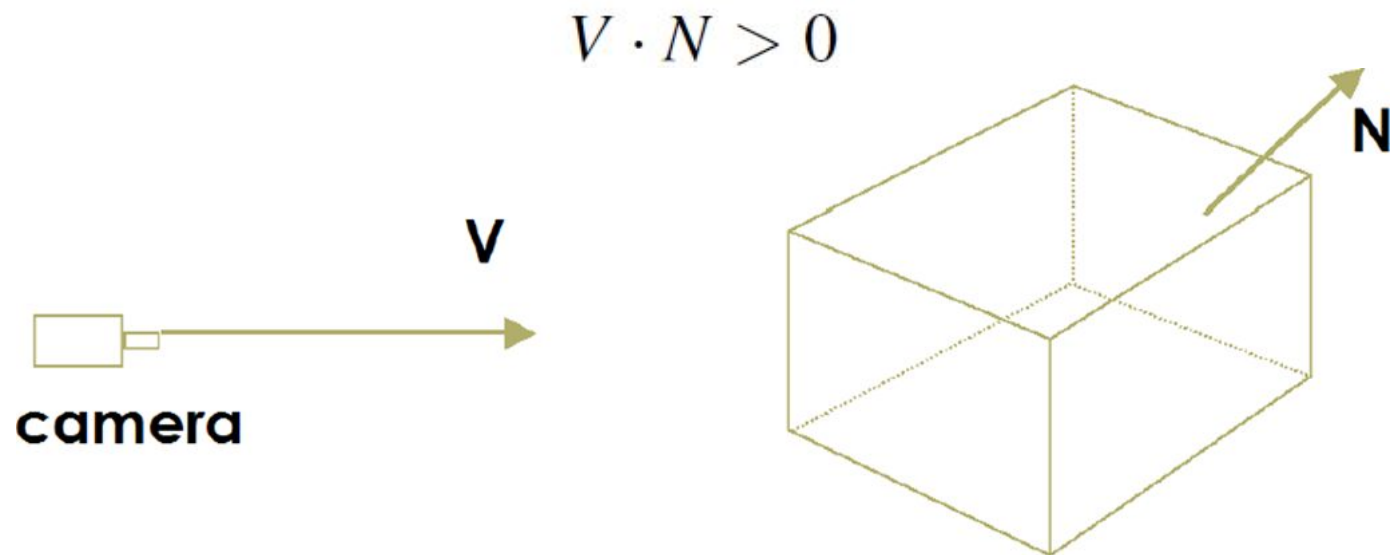
Back face culling

- Como descobrir quais são as “**faces de trás**”?
 - Usando vetores unitários



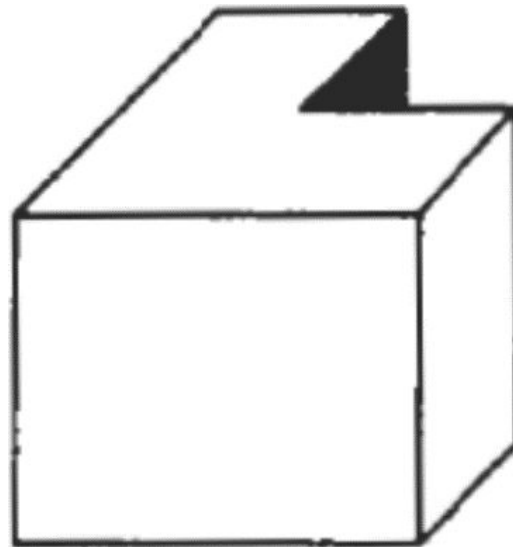
Back face culling

- Uma face é uma “face de trás” (**não visível**) de um polígono se o menor ângulo entre o vetor normal da face N e o vetor direção de observação V é menor do que 90°
 - Para calcular o ângulo entre os dois vetores, apenas coincide a origem do vetor V com a origem do vetor N



Back face culling

- Processo assume que cena é composta por objetos poliédricos convexos fechados



- **Se o objeto for côncavo, não funciona perfeitamente**, pois pode ter faces parcialmente visíveis e o algoritmo assume que ele está completamente visível

Back face culling

- Em geral, esse algoritmo é um passo inicial no processo de remoção de superfícies ocultas, pois simplifica significativamente a cena, permitindo uma renderização mais eficiente

```
glEnable (GL_CULL_FACE) ;  
glCullFace (GL_BACK) ;
```

- Com isso, restam apenas os polígonos/faces potencialmente visíveis para a câmera

Back face culling

- **Importante:** se ativar o back face culling mas não definir a normal da face, o OpenGL leva em consideração o sentido em que os vértices da face são definidos, para remover as faces ocultas
 - Faces desenhadas no sentido **anti-horário** são tratadas como **visíveis**
 - Enquanto as desenhadas no sentido **horário** são tratadas como **não visíveis**

Exemplo 1 - Back face culling

```
int main(int argc, char **argv){
    /* cria uma janela */
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB); /* single buffering */
    glutInitWindowSize(400, 400);
    glutCreateWindow("Exemplo 1 - Back Face Culling");

    glutDisplayFunc(display);

    init(); //executa funcao de inicializacao

    glutMainLoop(); //mostre tudo e espere
    return(0);
}
```

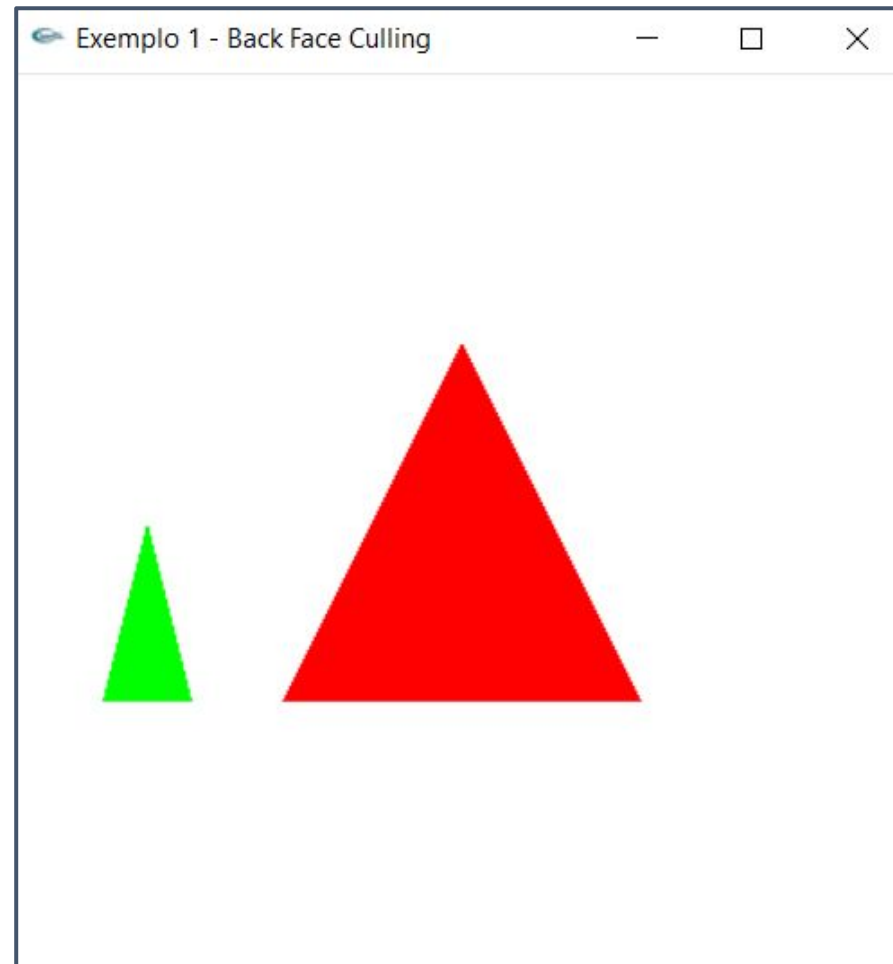
Exemplo 1 - Back face culling

```
void init(){  
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f); //define a cor de fundo  
  
    glMatrixMode(GL_PROJECTION); //define que a matriz é a de projeção  
    glLoadIdentity(); //carrega a matriz de identidade  
    glOrtho(-5.0, 5.0, -5.0, 5.0, -5.0, 5.0); //define uma projeção ortogonal  
}
```

Exemplo 1 - Back face culling

```
void display(void) {  
    //limpa o buffer  
    glClear(GL_COLOR_BUFFER_BIT);  
  
    //define que a matriz é a de modelo  
    glMatrixMode(GL_MODELVIEW);  
  
    glColor3f(1.0f, 0.0f, 0.0f); //N . V < 0 (visível)  
    glBegin(GL_TRIANGLES); //sentido anti-horário  
        glVertex3f(-2, -2, 0);  
        glVertex3f(2, -2, 0);  
        glVertex3f(0, 2, 0);  
    glEnd();  
  
    glColor3f(0.0f, 1.0f, 0.0f); //N . V > 0 (não visível)  
    glBegin(GL_TRIANGLES); //sentido horário  
        glVertex3f(-4, -2, -4);  
        glVertex3f(-3.5f, 0, -4);  
        glVertex3f(-3, -2, -4);  
    glEnd();  
  
    //força o desenho das primitivas  
    glFlush();  
}
```

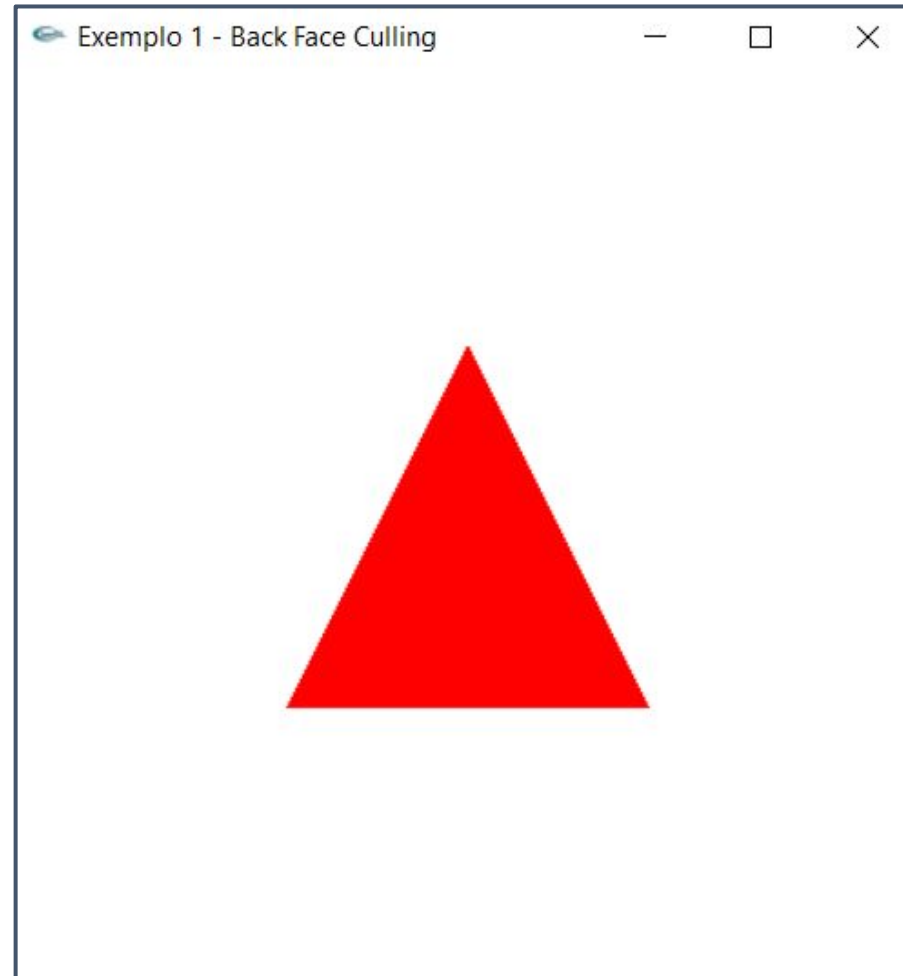
Exemplo 1 - Back face culling



Exemplo 1 - Back face culling

```
void init() {  
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f); //define a cor de fundo  
  
    //habilita remoção de faces ocultas  
    glEnable(GL_CULL_FACE);  
    glCullFace(GL_BACK);  
  
    glMatrixMode(GL_PROJECTION); //define que a matriz é a de projeção  
    glLoadIdentity(); //carrega a matriz de identidade  
    glOrtho(-5.0, 5.0, -5.0, 5.0, -5.0, 5.0); //define uma projeção ortogonal  
}
```

Exemplo 1 - Back face culling



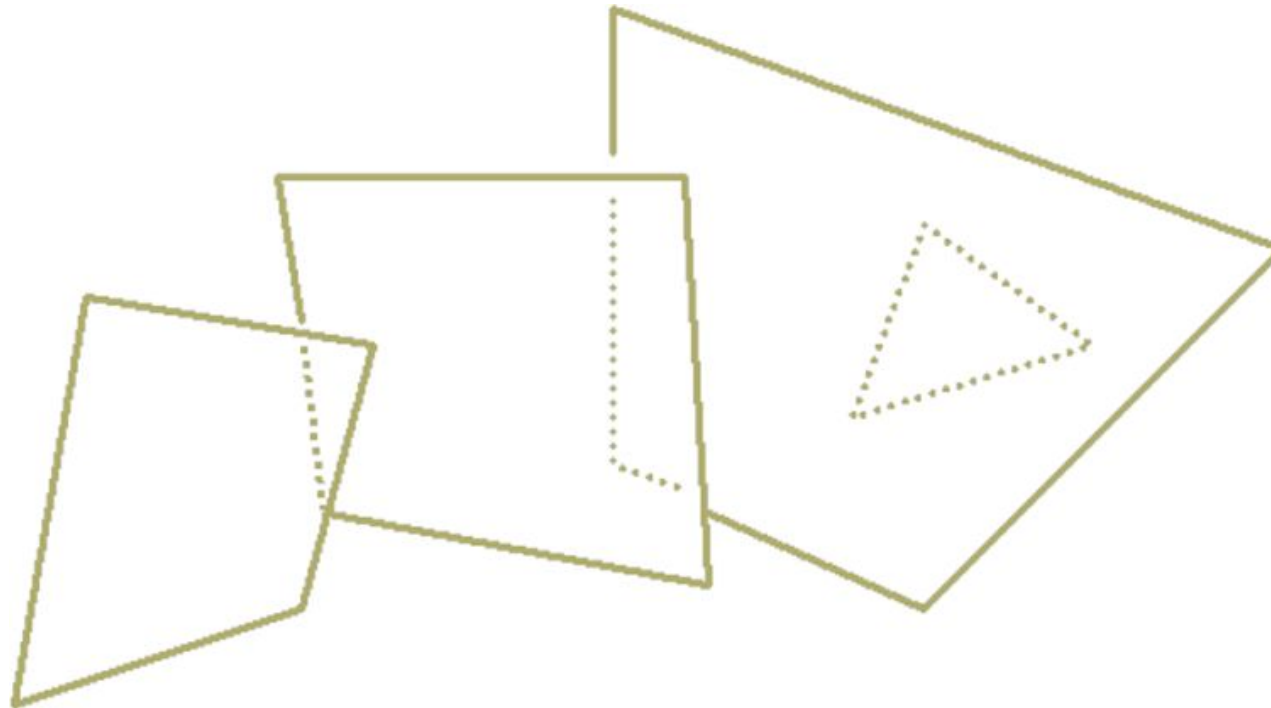
03

Algoritmo z-buffer

Programação OpenGL

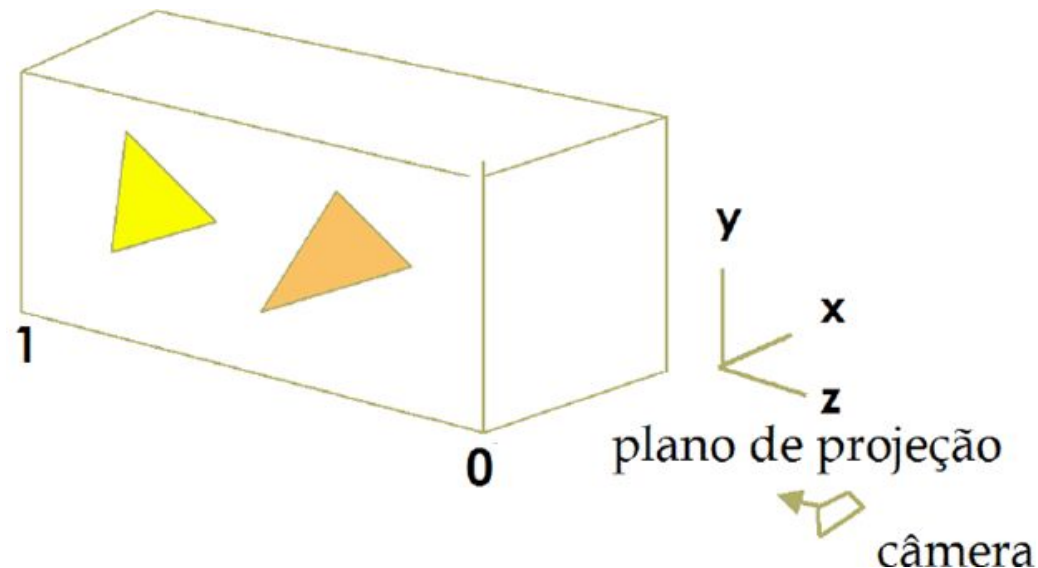
Outro problema: faces ocultas

- Algumas faces da cena ficam ocultas atrás de outras: só queremos desenhar (renderizar) as faces (ou partes delas) realmente visíveis



Solução: algoritmo z-buffer

- Considere que as faces passaram pela transformação de projeção (ortogonal e perspectiva), e tiveram suas coordenadas z armazenadas – suponha valores de z normalizados no intervalo 0 a 1
- Para cada pixel (x, y) , queremos traçar a face mais próxima da câmera, isto é, com menor valor de z



Algoritmo z-buffer

- Algoritmo usa dois buffers
 - **Frame buffer:** armazena os valores RGB que definem a cor de cada pixel
 - **Z-buffer:** mantém informação de profundidade associada a cada pixel
- Inicialização
 - Todas posições do z-buffer são inicializadas com a maior profundidade
 - Como o volume estará normalizado entre 0.0 e 1.0, então será 1.0
 - Todas as posições do frame buffer são inicializadas com a cor de fundo da cena

Algoritmo z-buffer

- A medida em que cada face é renderizada, são determinados os pixels através do algoritmo de scanline (ajuda a fazer o rendering)

```
// Configuração inicial
AtivarTesteDeProfundidade()
InicializarDepthBuffer()
InicializarFrameBuffer()

// Renderização dos objetos
Para cada objeto na cena:
    Para cada pixel a ser renderizado (x, y):
        Calcular o valor de profundidade do objeto no pixel (z_objeto)
        Obter o valor atual no depth buffer para o pixel (z_buffer)

        Se z_objeto < z_buffer:
            // O objeto está mais próximo da câmera, é visível
            Atualizar o depth buffer com o valor de profundidade do objeto (z_objeto)
            Calcular a cor do pixel com base nas fontes de luz, materiais, etc. (método de rendering de superfície)
            Atualizar o valor do pixel no frame buffer na posição (x, y) com a cor calculada

        Senão:
            // O objeto está mais distante da câmera, é oculto, descartar o pixel

// Saída
Exibir o conteúdo do frame buffer na tela
```

Algoritmo z-buffer

- Depois de todas as faces serem processadas, o depth buffer contém a **profundidade** das superfícies visíveis, e o frame buffer contém as **cores** dessas superfícies
 - Cena pronta para ser exibida está no frame buffer

Algoritmo z-buffer

- **Vantagens**

- Simples e comumente implementado em hardware
- Objetos podem ser desenhados em qualquer ordem
- Placas gráficas otimizam operações no z-buffer

- **Desvantagens**

- Quantidade de **memória** necessária (em um sistema 1280 x 1024 precisa de aproximadamente 1.3 milhões de posições). Também é preciso um depth buffer do mesmo tamanho
- Alguns **cálculos desnecessários**, devido a ordem arbitrária do processamento dos objetos

Algoritmo z-buffer

- Na função inicialização do GLUT, incluir uma requisição para criar um depth buffer (main). Por exemplo

```
glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB |  
                    GLUT_DEPTH) ;
```

- Habilitar z-buffer (init)

```
glEnable (GL_DEPTH_TEST) ;
```

- Ao gerar um novo quadro, o z-buffer deve ser limpo e inicializado (display)

```
glClear (GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT) ;
```

Exemplo 2 - Algoritmo z-buffer

```
int main(int argc, char **argv){  
    /* cria uma janela */  
    glutInit(&argc, argv);  
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB | GLUT_DEPTH); /* single buffering */  
    glutInitWindowSize(400, 400);  
    glutCreateWindow("Exemplo 2 - Z-Buffer");  
  
    glutDisplayFunc(display);  
  
    init(); //executa funcao de inicializacao  
  
    glutMainLoop(); //mostre tudo e espere  
    return(0);  
}
```

Exemplo 2 - Algoritmo z-buffer

```
void display(void) {  
    //limpa o buffer  
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);  
  
    //define que a matriz é a de modelo  
    glMatrixMode(GL_MODELVIEW);  
  
    //triângulo na frente  
    glColor3f(1.0f, 0.0f, 0.0f); //N . V < 0 (visível)  
    glBegin(GL_TRIANGLES); //sentido anti-horário  
        glVertex3f(-2, -2, 0);  
        glVertex3f(2, -2, 0);  
        glVertex3f(0, 2, 0);  
    glEnd();  
  
    //triângulo atrás  
    glColor3f(0.0f, 0.0f, 1.0f); //N . V < 0 (visível)  
    glBegin(GL_TRIANGLES); //sentido anti-horário  
        glVertex3f(0, 0, -2);  
        glVertex3f(4, 0, -2);  
        glVertex3f(2, 4, -2);  
    glEnd();  
  
    //força o desenho das primitivas  
    glFlush();  
}
```

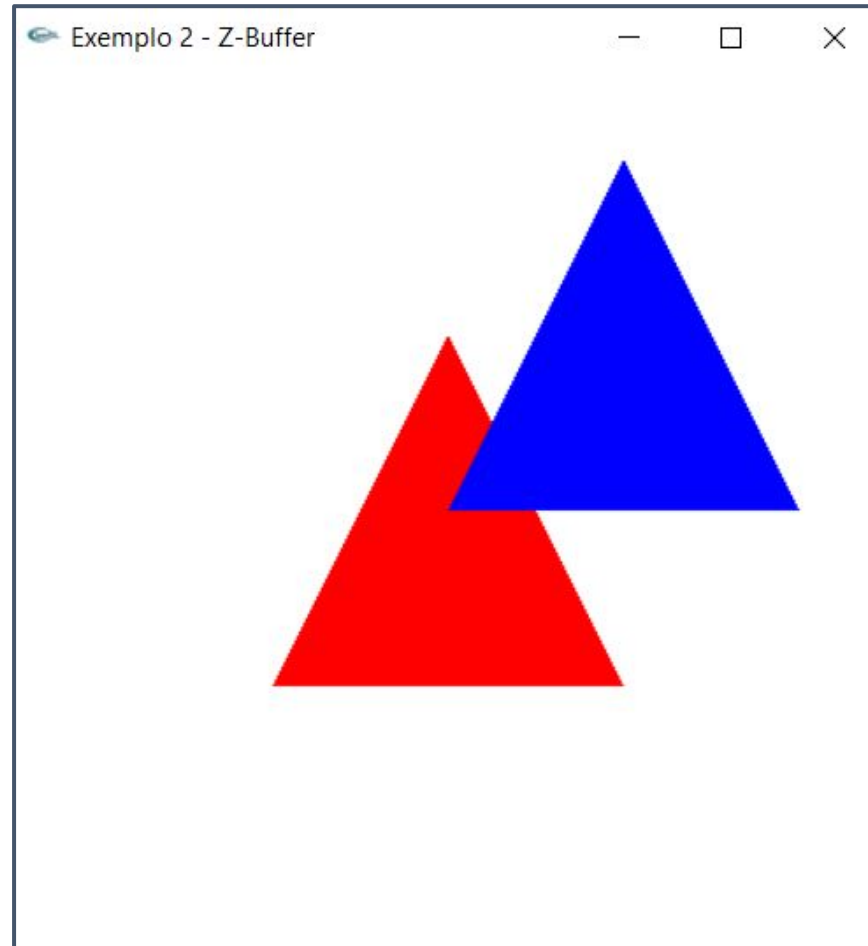
Exemplo 2 - Algoritmo z-buffer

```
void init(){
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f); //define a cor de fundo

    //habilita remoção de faces ocultas
    glEnable(GL_CULL_FACE);
    glCullFace(GL_BACK);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity(); //carrega a matriz de identidade
    glOrtho(-5.0, 5.0, -5.0, 5.0, -5.0, 5.0); //projeção ortogonal
}
```

Exemplo 2 - Algoritmo z-buffer



Exemplo 2 - Algoritmo z-buffer

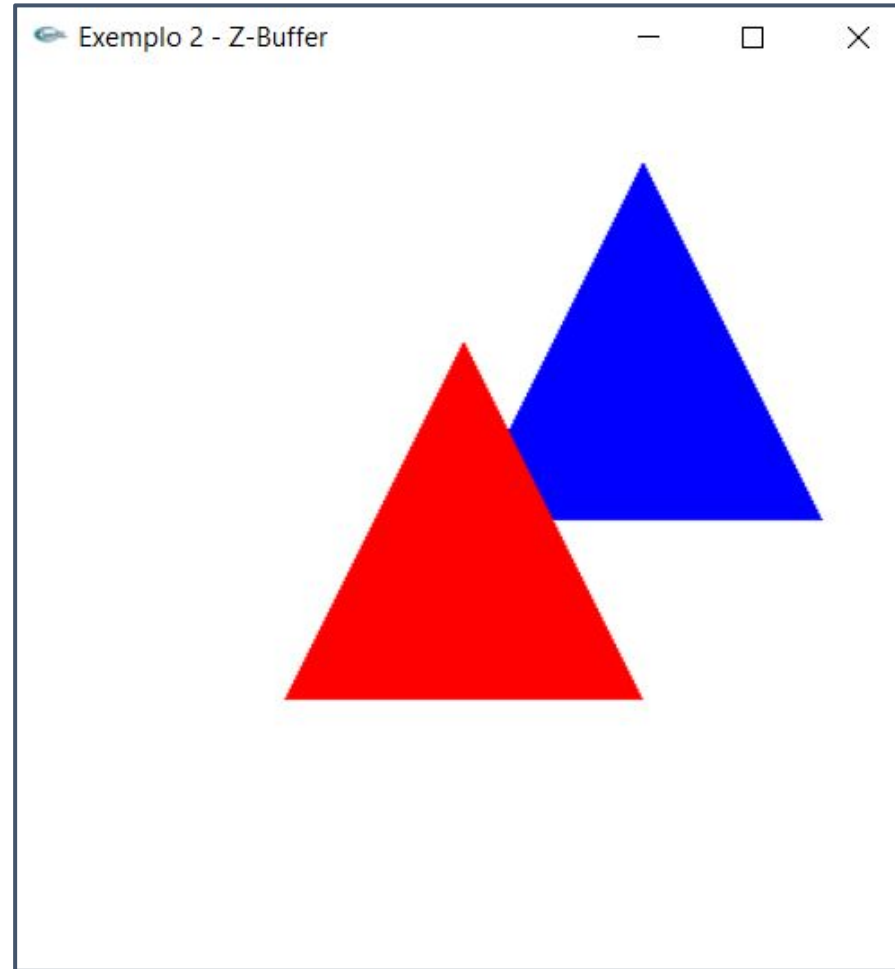
```
void init(){
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f); //define a cor de fundo

    //habilita o teste de profundidade
    glEnable(GL_DEPTH_TEST);

    //habilita remoção de faces ocultas
    glEnable(GL_CULL_FACE);
    glCullFace(GL_BACK);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity(); //carrega a matriz de identidade
    glOrtho(-5.0, 5.0, -5.0, 5.0, -5.0, 5.0); //projeção ortogonal
}
```

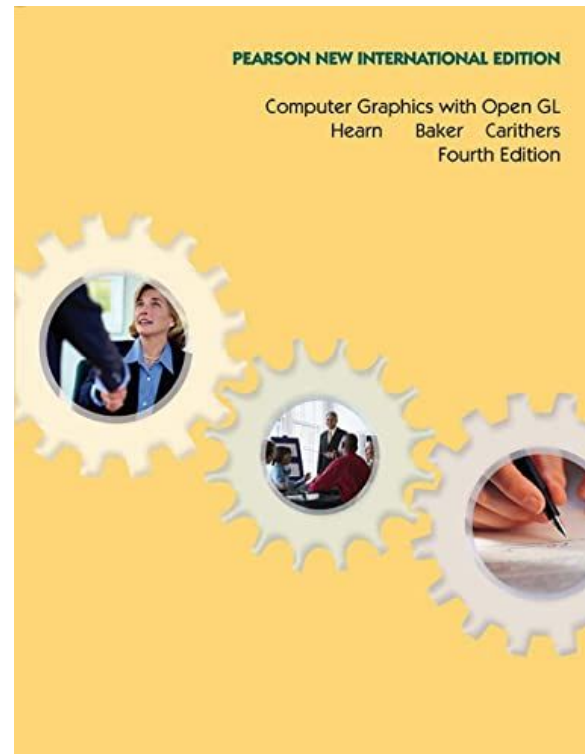
Exemplo 2 - Algoritmo z-buffer



Referências



AZEVEDO, Eduardo; CONCI, Aura.
Computação gráfica: teoria e prática.
Elsevier, 2003.



HEARN, DONALD. **Computer graphics with OpenGL.** Pearson Education Limited, 2014.
5º Edição.



PICHETTI, R. F; et al. **Computação Gráfica e Processamento de Imagens.**
Porto Alegre: SAGAH, 2022.

Referências



- ALENCAR, Aretha. **Notas de aula da disciplina Computação Gráfica da Universidade Tecnológica Federal do Paraná.** 2021.
- PAIVA, Anselmo. **Notas de aula da disciplina Computação Gráfica da Universidade Federal do Maranhão.** 2018.
- QUINTANILHA, Darlan. **Notas de aula da disciplina Computação Gráfica da Universidade Federal do Maranhão.** 2022.
- SILVA, Giovanni. **Notas de aula da disciplina Computação Gráfica da Universidade Federal do Maranhão.** 2019.